



FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

FEPZM · UASLP

PRÁCTICA 15

Uso de Protocolo Bluetooth LE en la Raspberry Pi Pico W

Materia: Microcontroladores

Docente: Prof. Ing. Jesús Padrón

Alumno: Carlos Ángel García Sifuentes

Semestre: 4° Semestre

Fecha: 18 de mayo de 2026



Uso de Protocolo Bluetooth LE en la Raspberry Pi Pico W para Enviar y Recibir Datos Inalámbricamente

CARLOS ÁNGEL GARCÍA SIFUENTES
a383917@alumnos.uaslp.mx

Docente: Ing. Jesús Padrón
Fecha: 18 de mayo de 2026

Resumen—En esta práctica se implementó un sistema de comunicación inalámbrica mediante el protocolo Bluetooth Low Energy (BLE) utilizando dos Raspberry Pi Pico W y la librería BTstack. Una de las placas fue configurada como servidor BLE, encargada de leer la temperatura interna del microcontrolador mediante el ADC y transmitirla inalámbricamente. La segunda placa fue configurada como cliente BLE, responsable de escanear, conectarse al servidor y mostrar la temperatura recibida en el monitor serial. El sistema funcionó correctamente, logrando lecturas de temperatura en tiempo real entre ambas placas de forma completamente inalámbrica.

I. INTRODUCCIÓN

El protocolo Bluetooth Low Energy (BLE) es un estándar de comunicación inalámbrica diseñado para operar con un consumo de energía muy bajo, lo que lo hace ideal para aplicaciones de monitoreo de sensores, dispositivos IoT y sistemas embebidos con alimentación por batería. A diferencia del Bluetooth clásico, BLE opera en 40 canales dentro de la banda de 2.4 GHz y soporta múltiples topologías de comunicación como punto a punto, broadcast y redes en malla [?].

La Raspberry Pi Pico W integra el chip CYW43439, que proporciona conectividad tanto WiFi como Bluetooth. Para implementar BLE en este microcontrolador se utiliza BTstack, una implementación de la pila Bluetooth desarrollada por BlueKitchen, diseñada para dispositivos embebidos con recursos limitados. BTstack se basa en un diseño de un solo hilo sin bloqueos, lo que lo hace eficiente y predecible en entornos de tiempo real.

En esta práctica se utilizan dos perfiles fundamentales de BLE: el perfil GAP (Generic Access Profile), que gestiona la detección y conexión entre dispositivos, y el perfil GATT (Generic Attribute Profile), que define cómo se organizan y transmiten los datos mediante servicios

y características. El servicio utilizado es el *Environmental Sensing Service*, un servicio estándar de Bluetooth para la transmisión de datos ambientales como temperatura, presión y humedad.

II. DESARROLLO

El proyecto se estructuró en dos ejecutables independientes compilados desde un mismo `CMakeLists.txt`: el servidor `picow_ble_temp_sensor` y el cliente `picow_ble_temp_reader`.

II-A Definición del servicio GATT

El archivo `temp_sensor.gatt` define los servicios y características que el servidor BLE ofrece. BTstack lo convierte automáticamente en un archivo de cabecera `.h` durante la compilación mediante la función `pico_btstack_make_gatt_header`.

Listing 1: `temp_sensor.gatt`

```
PRIMARY_SERVICE, GAP_SERVICE
CHARACTERISTIC, GAP_DEVICE_NAME, READ, "picow
temp"

PRIMARY_SERVICE, GATT_SERVICE
CHARACTERISTIC, GATT_DATABASE_HASH, READ,

PRIMARY_SERVICE,
    ORG_BLUETOOTH_SERVICE_ENVIRONMENTAL_SENSING
CHARACTERISTIC,
    ORG_BLUETOOTH_CHARACTERISTIC_TEMPERATURE,
    READ | NOTIFY | INDICATE | DYNAMIC,
```

II-B Servidor BLE

El servidor se compone de los archivos `server.c` y `server_common.c`. La función principal inicializa el driver CYW43, el ADC interno del chip, la pila L2CAP y SM,

y el servidor ATT. Un timer *heartbeat* actualiza la temperatura cada segundo y la envía mediante notificaciones GATT al cliente conectado.

La lectura del sensor interno se realiza en la función `poll_temp()`, que aplica la siguiente conversión:

Listing 2: Conversión de temperatura

```
float deg_c = 27.0f - (reading - 0.706f) /
    0.001721f;
current_temp = (uint16_t)(deg_c * 100);
```

El valor se multiplica por 100 para preservar dos decimales al enviarlo como entero de 16 bits. El cliente divide entre 100 al recibirlo para recuperar el valor en grados Celsius.

II-C Cliente BLE

El cliente implementa una máquina de estados en `client.c` que gestiona el ciclo de vida de la conexión BLE:

1. Escaneo de dispositivos cercanos.
2. Filtrado por el servicio *Environmental Sensing*.
3. Conexión al servidor detectado.
4. Descubrimiento del servicio y característica de temperatura.
5. Habilitación de notificaciones GATT.
6. Recepción y despliegue de temperatura en el monitor serial.
7. Reconexión automática en caso de desconexión.

El LED integrado de la RPi cliente indica el estado: parpadeo lento cuando no está conectado, y parpadeo rápido cuando está recibiendo datos.

II-D Compilación y flasheo

La compilación se realizó desde VS Code usando la extensión oficial de Raspberry Pi Pico con la opción **Build All Projects**, generando los archivos `.uf2` en la carpeta `build/`. Para flashear cada placa se mantuvo presionado el botón `BOOTSEL` al conectarla por USB y se copió el archivo correspondiente a la unidad de almacenamiento que aparece.

III. RESULTADOS

Al energizar ambas placas, el cliente escaneó los dispositivos BLE cercanos, identificó al servidor por su servicio *Environmental Sensing*, estableció la conexión y comenzó a recibir notificaciones de temperatura. En el monitor serial se observó la siguiente salida:

Listing 3: Monitor serial del cliente

```
BTstack up and running on XX:XX:XX:XX:XX:XX
Connecting to device with addr XX:XX:XX:XX:XX:XX
XX
read temp 34.53 degc
read temp 34.53 degc
read temp 35.00 degc
read temp 34.53 degc
```

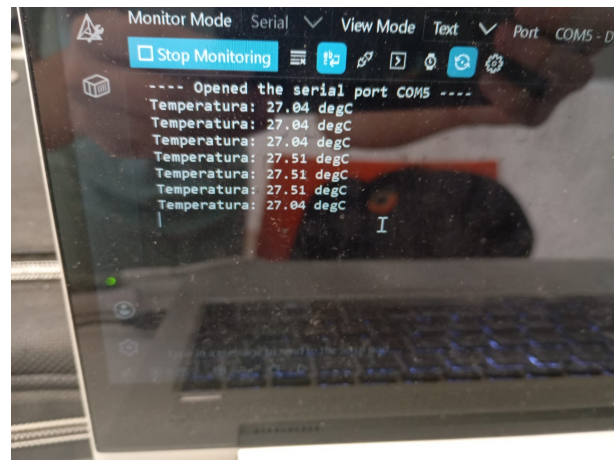


Figura 1: Monitor serial del cliente mostrando lecturas de temperatura en tiempo real.

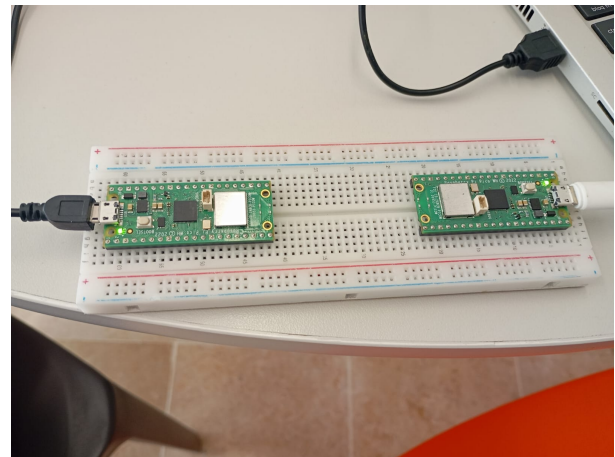


Figura 2: Montaje físico de las dos Raspberry Pi Pico W sobre protoboard.

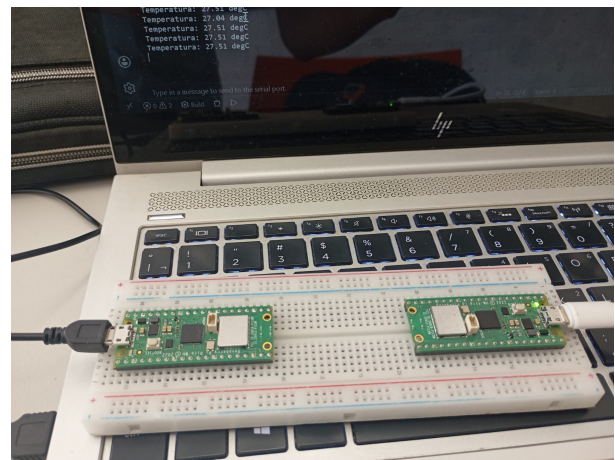


Figura 3: RPi Pico W actuando como servidor BLE con LED parpadeando.

IV. PREGUNTAS DE REPASO

1. ¿Cuál es la principal diferencia entre Bluetooth clásico y Bluetooth LE?

La diferencia principal radica en el consumo de energía y el tipo de aplicaciones. El Bluetooth clásico (BR/EDR) usa 79 canales y está optimizado para transmisión continua de datos a alta velocidad, como audio o transferencia de archivos. El BLE opera en 40 canales y está diseñado para envíos intermitentes de pequeñas cantidades de datos con consumo mínimo de energía, ideal para sensores y dispositivos IoT con batería.

2. ¿Qué ventajas ofrece Bluetooth LE frente a otras tecnologías inalámbricas?

BLE consume hasta 10 veces menos energía que WiFi, está integrado nativamente en todos los smartphones modernos sin requerir hardware adicional, soporta múltiples topologías de red, y ofrece funciones de posicionamiento mediante RSSI y ángulo de llegada (AoA). Esto lo hace superior a otras tecnologías como Zigbee en términos de compatibilidad con dispositivos móviles.

3. ¿Cuál es el propósito del archivo `btstack_config.h`?

Es el archivo de configuración en tiempo de compilación de BTstack. Define qué características del stack se habilitan, como el modo periférico (`ENABLE_LE_PERIPHERAL`) o central (`ENABLE_LE_CENTRAL`), los tamaños de buffers ACL y SCO, el control de flujo HCI para evitar saturar el bus del CYW43, y el número máximo de conexiones y entradas en la base de datos de dispositivos.

4. ¿Qué ventajas tiene BLE en un sistema de monitoreo frente a WiFi o cable?

Frente a WiFi, BLE consume mucho menos energía y no requiere infraestructura de red. Frente al cable, elimina el cableado físico, reduce costos de instalación, permite mayor movilidad de los sensores y facilita escalar el sistema agregando más nodos de forma inalámbrica.

5. ¿Por qué es importante la reconexión automática del cliente BLE?

En aplicaciones reales, las desconexiones ocurren por interferencias, distancia o cortes de energía. Sin reconexión automática, el sistema dejaría de funcionar hasta una intervención manual. La reconexión garantiza continuidad del servicio y hace el sistema adecuado para entornos industriales o remotos sin supervisión constante.

6. En un ambiente con muchos dispositivos BLE, ¿qué estrategias evitan interferencias?

La estrategia usada en esta práctica es el filtrado por UUID de servicio, conectándose solo a dispositivos con el servicio *Environmental Sensing*. Otras estrategias incluyen filtrar por nombre o dirección MAC del servidor, y aprovechar que BLE ya implementa salto de frecuencia (FHSS) que reduce naturalmente las interferencias.

7. ¿Cómo modificarías esta práctica para mostrar datos en una app móvil?

Se podría usar una app genérica de BLE como nRF Connect para conectarse sin desarrollo adicional. Para una app personalizada, se desarrollaría en Flutter o React Native usando las APIs BLE del sistema operativo para

escanear, conectarse y suscribirse a las notificaciones de temperatura. El servidor no requeriría modificaciones ya que implementa el perfil GATT estándar.

V. CONCLUSIONES

Se implementó con éxito un sistema de comunicación BLE entre dos Raspberry Pi Pico W, logrando transmitir lecturas de temperatura en tiempo real de forma completamente inalámbrica. Se comprendió la arquitectura de BTstack basada en máquinas de estados y el flujo de eventos asíncronos, así como la estructura del perfil GATT para la definición de servicios y características.

Durante el desarrollo se enfrentaron dificultades como errores en los nombres de archivos, pérdida de caracteres especiales al copiar código, y ausencia de archivos de configuración. Estas situaciones permitieron desarrollar habilidades de diagnóstico y resolución de errores de compilación en proyectos embebidos con múltiples archivos.

Como mejora futura, se podría extender el sistema para transmitir el estado de botones físicos y controlar actuadores como LEDs en la placa remota, ampliando las aplicaciones del sistema BLE. También sería valioso integrar una aplicación móvil para visualizar los datos de forma más amigable, aprovechando que el servidor ya implementa el perfil GATT estándar de *Environmental Sensing*.